O código do totem

Como os 5 padrões do Grupo 3 viraram classes Java

Abstract Factory · Decorator · Command · Mediator · Memento

Grupo 3 · Sala de Aula Invertida · Design Patterns (GoF) · Java 17

Um pacote para cada padrão

22 classes · 5 pacotes · nenhuma biblioteca externa

`abstractfactory`

Abstract Factory

9 classes

`decorator`

Decorator

3 classes

`command`

Command

6 classes

`memento`

Memento

2 classes

`mediator`

Mediator

6 classes

sorveteria — java

```
$ javac -encoding UTF-8 -d out $(find src -name "*.java")
$ java -cp out sorveteria.Main
```

Main.java

simula o cliente usando o totem; cada arquivo marca em comentário onde o padrão começa e termina

Abstract Factory

CRIACIONAL

Cria a família de itens de uma linha — sorvete e casquinha saem sempre do mesmo tipo

01 A fábrica cria dois produtos. Com um só, seria Factory Method.

02 O resto do código nunca escreve new SorveteZero(...).

03 Trocar a linha = trocar a fábrica: nada mais muda.

```
public interface FabricaLinha {
    Sorvete criarSorvete(String sabor);
    Casquinha criarCasquinha();    // 2º produto
}

public class FabricaZero
        implements FabricaLinha {

    public Sorvete criarSorvete(String sabor) {
        return new SorveteZero(sabor,
                               criarCasquinha());
    }

    public Casquinha criarCasquinha() {
        return new CasquinhaZero();
    }
}
```

No código: FabricaLinha declara os dois produtos; cada fábrica concreta entrega a versão da sua linha.

Decorator

ESTRUTURAL

Cada cobertura embrulha o sorvete e soma descrição e preço

01 Implementa Sorvete e guarda um Sorvete: por isso empilha.

02 A classe SorveteZero não muda nada para ganhar cobertura.

03 Sem o padrão: uma classe para cada combinação.

```
public abstract class CoberturaDecorator
        implements Sorvete {    // mesma interface

    protected final Sorvete sorvete;  // o embrulhado
}

public class CaldaChocolate
        extends CoberturaDecorator {

    public String getDescricao() {
        return sorvete.getDescricao() + " + calda";
    }

    public double getPreco() {
        return sorvete.getPreco() + 2.50;
    }
}

new Granulado(new CaldaChocolate(sorvete))
```

No código: o decorator implementa Sorvete e guarda um Sorvete — é isso que permite empilhar uma cobertura na outra.

Command

COMPORTAMENTAL

Cada ação do cliente vira um objeto com executar() e desfazer()

01 O comando não tem regra de negócio: ele chama o Pedido.

02 Três comandos: Adicionar, Remover e Aplicar cupom.

03 HistoricoComandos guarda os comandos numa pilha.

```
public interface Comando {
    void executar();
    void desfazer();
}

public abstract class ComandoPedido
        implements Comando {
    protected final Pedido pedido;   // receptor
    protected SnapshotPedido backup; // memento
}

public class AdicionarItemCommand
        extends ComandoPedido {

    public void executar() {
        salvarBackup();          // foto antes de mudar
        pedido.adicionar(item); // o trabalho é do Pedido
    }
}
```

No código: quem altera o pedido é a linha pedido.adicionar(item) — o comando só organiza a ação.

Memento

COMPORTAMENTAL

O Pedido salva e restaura o próprio estado sem expor o que tem dentro

01 Getters sem public: só o Pedido, no mesmo pacote, lê.

02 O snapshot copia a lista, então não muda depois de criado.

03 Quem guarda o snapshot é o comando; quem lê é o Pedido.

```
public class Pedido {                    // Originator

    public SnapshotPedido criarSnapshot() {
        return new SnapshotPedido(itens, desconto);
    }

    public void restaurar(SnapshotPedido s) { ... }
}

public class SnapshotPedido {            // Memento

    List<Sorvete> getItens() { ... }  // package-private
    double getDesconto()     { ... }   // só o Pedido lê
}
```

No código: os getters do snapshot não são public, então o estado salvo fica fechado dentro do pacote.

Mediator

COMPORTAMENTAL

Os painéis do totem não se conhecem: todos falam só com o mediador

01 Nenhum painel tem referência a outro painel.

02 O mediador coordena; quem executa é o comando e o Pedido.

03 Adicionar um painel novo não mexe nos que já existem.

```
public class PainelCardapio extends Componente {

    public void selecionar(String sabor,
                           String... cob) {
        mediator.notificar(this, "adicionar");
    }   // não conhece carrinho nem total
}

public class TotemMediator implements Mediator {

    public void notificar(Componente de,
                          String evento) {
        // escolhe a fábrica, monta as coberturas,
        // cria o comando e manda os painéis atualizarem
    }
}
```

No código: o único atributo de comunicação do painel é o mediator, herdado de Componente.

O coração do desfazer

É aqui que Command e Memento se encontram

```
protected void salvarBackup() {
    this.backup = pedido.criarSnapshot();
}

public void desfazer() {
    if (backup != null) pedido.restaurar(backup);
}
```

01 O comando tira a foto do pedido antes de agir

02 O histórico empilha o comando executado

03 O desfazer desempilha e manda restaurar a foto

O comando não sabe desfazer sozinho: ele desfaz restaurando o memento que guardou antes de agir.

O que a Main executa

O roteiro segue o diagrama de sequência, do primeiro clique até o pagamento

01	Escolher a linha Zero	Abstract Factory
02	Montar sorvete com calda e granulado	Decorator
03	Adicionar ao pedido	Command + Memento
04	Aplicar cupom	Command
05	Desfazer o cupom e o item	Memento
06	Remover item e desfazer de novo	Command + Memento
07	Finalizar o pedido	Mediator

A saída no terminal

Trecho real do programa — o desfazer devolve o pedido ao estado anterior

sorveteria — java

```
=== 2. Montar sorvete com coberturas (Abstract Factory + Decorator) ===
[Cardapio] cliente montou: morango [calda, granulado]
[Carrinho] itens:
   1. Sorvete zero acucar de morango na casquinha integral sem acucar
      + calda de chocolate + granulado (R$ 17.50)
[Total] subtotal R$ 17.50 | desconto 0% | total R$ 17.50

=== 4. Aplicar cupom (Command) ===
[Total] subtotal R$ 31.00 | desconto 10% | total R$ 27.90

=== 5. Desfazer o cupom (Memento) ===
[Carrinho] cliente clicou em DESFAZER
>> Desfeito: Aplicar cupom SORVETE10
[Total] subtotal R$ 31.00 | desconto 0% | total R$ 31.00
```

Abstract Factory cria · Decorator enfeita · Mediator coordena · Command executa · Memento desfaz

22 classes, 5 pacotes, um cenário só: o totem da sorveteria.

Abstract Factory · Decorator · Command · Mediator · Memento